ASSIGNMENT 4-CO4

NAME	GAYATHRI.P
REG NO	192424324
CORSE	Mobile Computing
CODE	ITA0329

Task 1 – Architecture Review

1.1 Diagnose the description

The draft note has seven technical deficiencies; each one leaves a reader unable to build or test the system.

#	Deficiency in the note	Why another engineer cannot build or evaluate from it
1	Only three layers (hardware, kernel, applications). Drivers, hardware abstraction layer (HAL), native libraries and framework services are missing.	Nobody can tell where vendor camera or sensor code lives, what is portable across devices, or which layer owns system services such as activity or power management.
2	"A kernel runs the applications" gives no kernel type and a wrong duty. A kernel does not run apps; it schedules processes, manages virtual memory, mediates system calls and loads drivers.	Monolithic (Linux), hybrid (XNU/Darwin) and Windows NT kernels differ in fault isolation and driver placement. Without the type, driver crashes and latency cannot be reasoned about.
3	"Compiled and executed by the runtime"	Start-up time, memory footprint and

	does not say what the runtime executes (bytecode or native code) or when translation happens (install, load or run time).	portability all depend on interpreter vs JIT vs ahead-of-time compilation. The note allows none of these to be predicted.
4	"Permissions" does not say who grants them (user or installer), when (install or run time) or where they are enforced (framework check or kernel UID/SELinux).	A security reviewer cannot locate the enforcement point, so a bypass cannot be tested for or ruled out.
5	"Saves battery when needed" gives no trigger (screen off, idle, 15% charge), no mechanism (frequency scaling, suspend, deferred jobs) and no owning component.	The behaviour is untestable: no condition can be set up and no expected result stated.
6	"Developers use the framework" does not define the boundary between an application and the OS, nor the path across it (API call, Binder IPC, system call).	The trust boundary, the cost of crossing it and the point where checks run are all unknown.
7	No process model: sandboxing, app lifecycle and low-memory policy are absent.	The reader cannot say what happens to a background app under memory pressure or how one app is stopped from reading another's data.

1.2 Repaired specification – Android

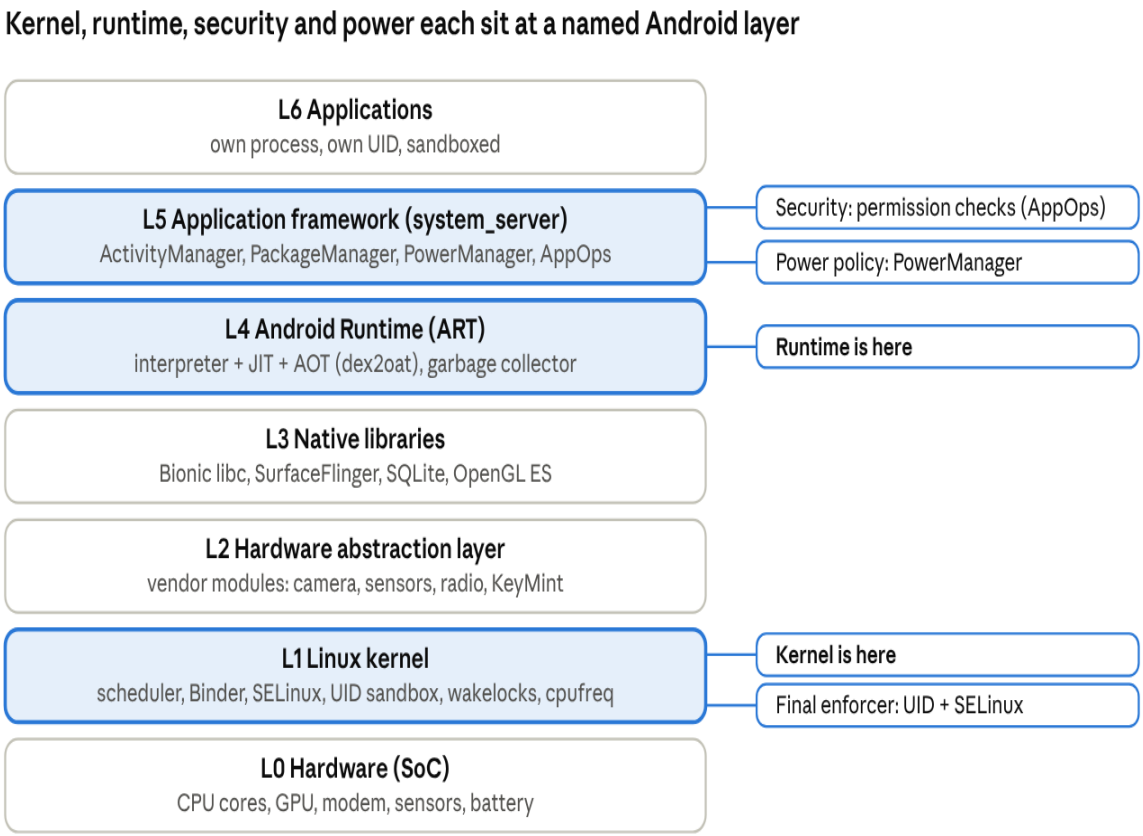
Android is chosen. The table lists each layer from the bottom up with its name, responsibility, what it exposes to the layer above and what it depends on below.

Layer	**Responsibility**	**Exposes to the layer above**	**Depends on (below)**
L0 Hardware	Application CPU cores, GPU, modem,	Registers,	–

(SoC)	sensors, battery and power-management IC; raises interrupts.	memory-mapped I/O, interrupts	
L1 Linux kernel (monolithic, loadable drivers)	Process scheduling (CFS), virtual memory, Binder IPC driver, SELinux (mandatory access control), per-app Linux UID sandbox, kernel power management (wakelocks, cpufreq/DVFS, suspend), device drivers. **Kernel is located here.**	System calls, /dev/binder, /proc, /sys	L0
L2 Hardware Abstraction Layer	Vendor-implemented modules for camera, audio, sensors, radio, GPS behind fixed interfaces (HIDL/AIDL).	Stable hardware interfaces	L1 drivers via device nodes and ioctl
L3 Native libraries	Bionic libc, SurfaceFlinger (compositor), OpenGL ES/Vulkan, SQLite, media codecs, WebView.	C/C++ APIs, JNI and NDK	L1 system calls, L2 HAL
L4 Android Runtime (ART)	Executes DEX bytecode: interpreter, JIT, profile-guided ahead-of-time compilation (dex2oat), garbage collection. **Runtime is located here.**	Managed execution environment, JNI bridge	L3 Bionic, L1
L5 Application framework (system_server)	ActivityManagerService (lifecycle, process priority), PackageManagerService (install, signature check), PowerManagerService (wake and doze policy), WindowManager, permission checks via AppOps. **Permission enforcement point (framework side) and**	Public Java/Kotlin APIs: Activity, Service, Intent,	L4, Binder (L1)

	power-management policy are located here.	permission APIs	
L6 Applications	User and system apps; each runs in its own process and UID, inside a sandbox.	UI and services to the user	L5 APIs only

Key locations: kernel = L1; runtime = L4; security enforcement = L5 for permission checks and L1 for the UID sandbox and SELinux (the kernel is the final enforcer); power management = policy in L5 (PowerManagerService) and mechanism in L1 (wakelocks, cpufreq, suspend). The boundary between an application and the OS is the Binder call into system_server (L5) and the system-call interface (L1).



Android layered architecture · 7 layers, 4 located components

The shaded layers hold the four components the task asks to locate: the kernel at L1, the runtime at L4, and the permission check and power policy at L5.

1.3 Design defence

The rewritten specification is precise because every claim is testable. Each layer has a named responsibility, a stated interface and a stated dependency, so another engineer can build or check it. The kernel type, runtime, permission enforcement point and power mechanism each sit at a named layer. It uses measurable terms such as Binder, UID sandbox and dex2oat, not vague words like "secure" or "optimised". The application–OS boundary and its communication path are explicit. A marketing overview could not be validated against a running device; this specification can.

Task 2 – Platform Requirement to Architectural Comparison

2.1 Model the requirement

Element	**Statement**
Input (what is compared)	Android (Linux kernel, ART), iOS (Darwin/XNU, Objective-C runtime) and Windows (NT kernel, WinRT) as hosts for a secure messaging app.
Output	A decision table showing, per platform, whether a message arrives while the app is not running, how it is decrypted, and what the app must do differently.
Objective	Deliver every end-to-end-encrypted message within a few seconds without keeping the CPU awake.
Constraints	Background execution (the app may be suspended or killed); security (message content must be decrypted only in the app's sandbox); battery (no permanent wakelock or polling); responsiveness (notification

	within seconds of arrival).
Decision	Which platform-specific delivery path to build for each OS (push-triggered wake-up vs persistent connection), and what shared code can be reused.
Architectural information needed	Kernel structure and driver model; runtime and when code becomes machine code; process model and background rules; sandbox and IPC mechanism; the push service and who wakes the app.

2.2 Hypothesis

The process and background-execution model (what the OS allows a suspended or terminated app to do, and who wakes it) will affect background message delivery more than kernel structure, because all three kernels deliver the network interrupt equally well, but only the OS policy above the kernel decides whether app code may run on arrival.

2.3 Comparison matrix

Aspect	**Android**	**iOS**	**Windows**	**Consequence for the messaging app**
Kernel family and structure	Linux, monolithic; drivers are loadable modules running in kernel space.	Darwin XNU, hybrid: Mach microkernel core + BSD layer + IOKit drivers in one address space.	Windows NT, hybrid: executive + microkernel + HAL; drivers run in kernel mode.	The network-interrupt path is equivalent. A bad driver can crash all three; the app is unaffected by the difference.
Kernel-level programming interface	System calls through Bionic libc; NDK (C/C++) and JNI from managed code.	BSD syscalls and Mach traps via libSystem; IOKit for drivers; Swift/Objective-C and C.	NT native API (ntdll syscalls) under Win32; WDM/WDF drivers; WinRT components in C++.	Native crypto code in C/C++ is shareable between Android (NDK) and Windows (C++/WinRT); iOS needs a

				separate build of the same C library.
Runtime environment	ART: DEX bytecode run by interpreter, JIT and profile-guided AOT (dex2oat); concurrent copying garbage collector.	Objective-C runtime: native ARM binary, dynamic dispatch via objc_msgSend, ARC reference counting; no JIT for third-party apps.	WinRT: COM-based ABI projected to C++/C#/JavaScript; C# store apps are compiled ahead-of-time (.NET Native) with garbage collection.	Android needs warm-up (JIT/profile) and GC pauses; iOS has deterministic memory release; Windows depends on language projection. Cold-start of a push-woken process is fastest on iOS (no bytecode to load).
Application framework	Android SDK: Service, WorkManager, Firebase Cloud Messaging (FCM) client in Google Play services.	Cocoa Touch: UIKit, UserNotifications, BackgroundTasks; APNs client in the apsd daemon.	WinRT/UWP APIs, background tasks, Windows Push Notification Services (WNS).	Three separate push integrations are required; none of the three lets the app keep its own socket alive in the background.
Application isolation model	One Linux UID per app, an SELinux domain and a seccomp filter.	Seatbelt sandbox container per app, mandatory code signing and entitlements.	AppContainer: low-integrity process with capability SIDs.	Message database and keys sit in app-private storage on all three; a Notification Service Extension (iOS) or background task (Windows) must be granted access to the same key.
Inter-process communication	Binder: a kernel driver; each transaction	Mach ports and XPC services; the kernel checks port	ALPC and COM/RPC, brokered by the system.	Push arrives through an IPC hop from a system service to the app

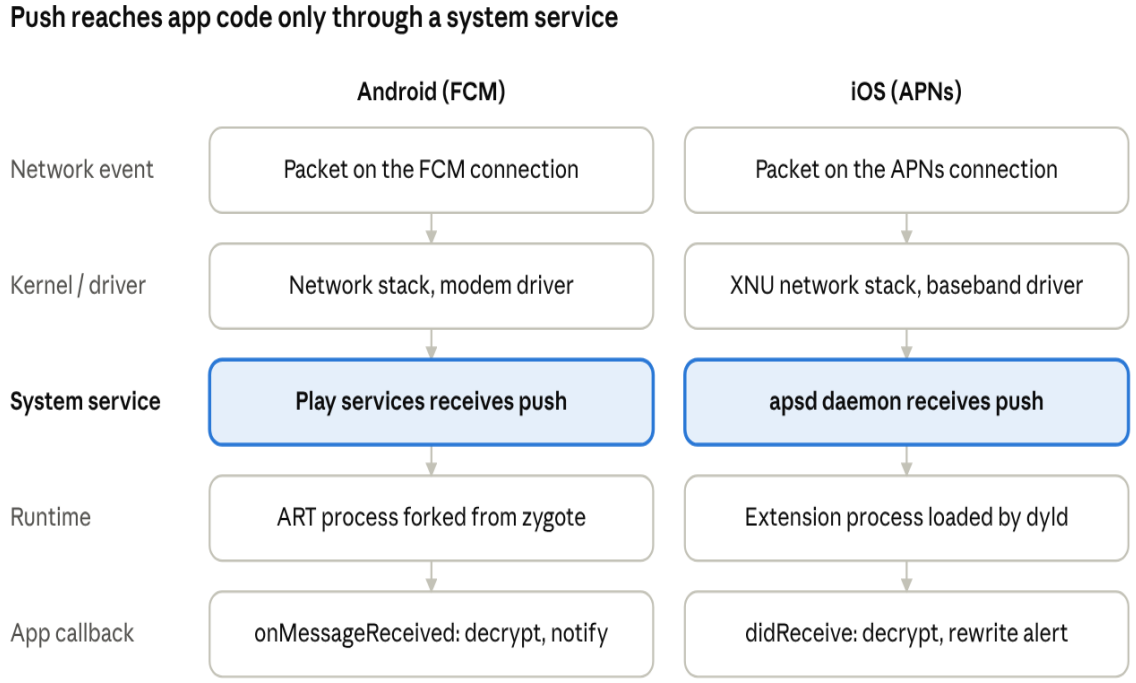
	carries the caller's UID for permission checks.	rights.		on every platform; the app cannot trust the sender except through the OS-provided identity.

2.4 Desk-check: one incoming message with the screen off

Two platforms are traced, Android and iOS, layer by layer.

Layer	**Android (FCM high-priority message)**	**iOS (APNs with mutable-content)**
Hardware / network event	Modem receives a packet on the persistent Google connection and wakes the application processor.	Modem receives a packet on the persistent Apple push connection and wakes the application processor.
Kernel / driver	Linux network stack and modem driver deliver the packet; kernel wakelock held while the packet is handled.	XNU network stack and baseband driver deliver the packet.
System service	Google Play services (FCM client, a persistent process) receives the push; ActivityManager may start the app's process as an allowed high-priority exception to Doze.	apsd daemon receives the push and hands it to the UserNotifications daemon.
Runtime	ART starts a new process through zygote fork, if the app is not running.	The system launches a Notification Service Extension process (native binary, loaded by dyld).

Application callback	FirebaseMessagingService.onMessageReceived runs; the app decrypts the message with its Keystore key and posts the notification.	The extension's didReceive callback runs; it decrypts using a key shared through the keychain access group and rewrites the notification.



message path with screen off · Android vs iOS, 5 layers

The highlighted row is where the platforms differ: the system service, not the kernel, decides whether and how app code runs.

2.5 Challenge the comparison

Changed condition: the application has been terminated by the OS because of memory pressure.

1. **Prediction.** Delivery differs. On Android the message still arrives and the app is restarted. On iOS a visible notification still arrives, but a silent background wake-up of the full app is not guaranteed.

2. **Responsible element.** Android: lmkd (the low-memory killer daemon) removes the process, but the push client is in a separate Google Play services process, so ActivityManager can start the app again. iOS: jetsam terminated the app, and relaunch for background work is at the discretion of the OS scheduler; only the extension process, with a short time budget, is allowed to run for a visible notification.
3. **Trace of the relevant steps.** Android: lmkd kills app → FCM message arrives in Play services → ActivityManager forks a new app process from zygote → onMessageReceived runs. iOS: jetsam kills app → apsd receives push → the notification extension is launched, decrypts and posts the alert; the main app is not launched. The prediction was correct. The decisive architectural element is the process model (who owns the push channel and who may start a process), not the kernel.

Task 3 – Runtime Environment Analysis

3.1 Execution model

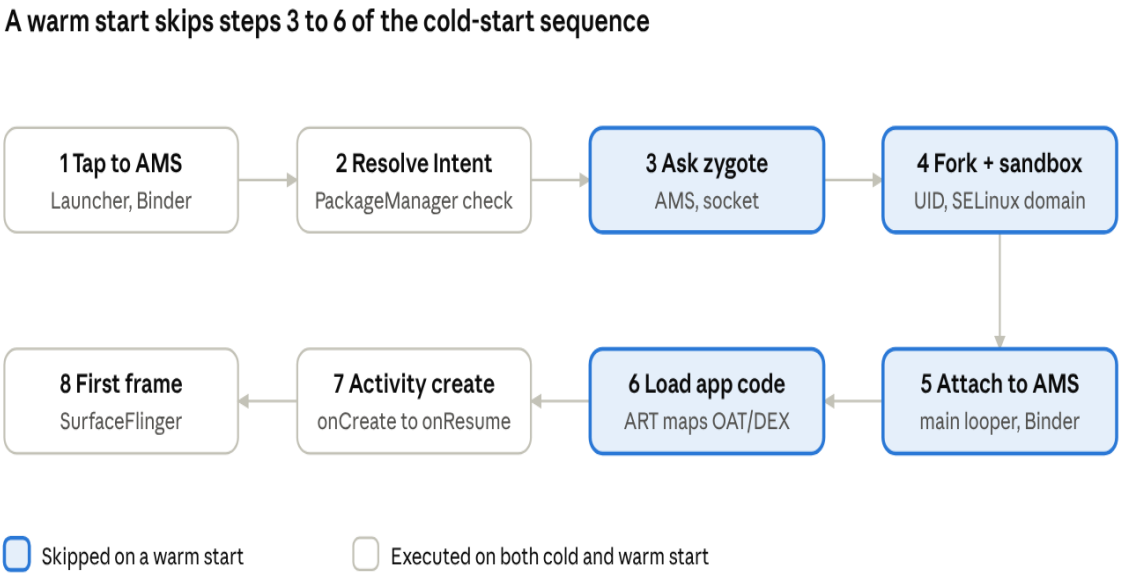
Platform	Code delivered to the device	Who loads it	When it becomes machine instructions
Android (Dalvik, and ART which replaced it)	APK containing DEX bytecode (classes.dex), plus optional native .so libraries.	The process forked from zygote; the ART/Dalvik class loader reads the DEX/OAT files.	Dalvik: at run time, by interpreter plus JIT. ART: partly at install or idle time (dex2oat produces native OAT code), hot methods by JIT at run time, the rest interpreted.
iOS (Objective-C runtime)	IPA containing a signed native ARM64 Mach-O binary, compiled ahead of time by the developer toolchain.	The kernel executes the binary and dyld (dynamic linker) maps it and its libraries; the Objective-C runtime registers classes.	At load time: pages are mapped and signature-checked as they are paged in. No JIT is allowed for third-party apps.
Windows (WinRT)	APPX package: native C++	The Windows loader creates the process;	Native C++: at load time. Managed store apps: at install

	executable, or managed code as IL which the store compiles to native (.NET Native).	the WinRT activation layer instantiates runtime classes.	time (ahead-of-time compiled); managed IL not pre-compiled: at run time by JIT.

3.2 Launch sequence – Android cold start

#	Step and its purpose	Performed by	Must already exist
1	The tap becomes a startActivity request. Purpose: turn user intent into a system request across the process boundary.	Launcher app, sent by Binder to ActivityManagerService (AMS)	system_server running, Launcher process alive
2	The Intent is resolved to a component and permission-checked. Purpose: make sure the target exists and the caller may start it.	AMS with PackageManagerService	The package installed and its signature verified
3	No process exists for the app, so AMS asks zygote to start one. Purpose: create an isolated process quickly.	AMS, zygote (socket command)	Zygote alive with preloaded framework classes
4	Zygote forks. The child inherits ART and the preloaded classes (copy-on-write) and then switches to the app's UID and SELinux domain. Purpose: fast start with the sandbox applied.	zygote, Linux kernel (fork, setuid)	Kernel process and memory services; app UID assigned
5	ActivityThread.main() starts the main looper and attaches to AMS by Binder. Purpose: register the new process so AMS can send it commands.	App process, Binder driver	Binder driver, main thread looper
6	bindApplication: the APK is opened, the class loader created, compiled code (OAT) or DEX is mapped and the Application object is constructed. Purpose: make app code reachable by the runtime.	ART in the app process	APK, DEX/OAT files, runtime initialised

7	AMS schedules the launch Activity: onCreate, onStart, onResume run. Purpose: let app code build its UI state.	Framework callbacks in the app process	Application object created (step 6)
8	The view hierarchy is measured and drawn, the window is registered and SurfaceFlinger composes the first frame at the next vsync. Purpose: show the first screen.	ViewRootImpl, WindowManager, SurfaceFlinger, display HAL	Activity resumed, GPU/display HAL available



Android cold-start launch sequence · 8 steps

The flow runs left to right on the top row, then right to left on the bottom row; the shaded steps create the process and load the runtime, which a warm start already has.

3.3 Compare runtime strategies

Aspect	**Android (ART / Dalvik)**	**iOS (Objective-C runtime)**	**Windows (WinRT / .NET)**
Interpretation, JIT, AOT	Dalvik: interpreter + trace JIT. ART: interpreter + JIT for hot code + profile-guided AOT (dex2oat).	AOT only; code is native at build time.	.NET Native: AOT at store; CLR on older stack: JIT.

Memory management	Tracing garbage collector (concurrent copying). Needs spare heap headroom and causes short pauses that can drop frames.	Reference counting (ARC). Memory is freed at the moment the last reference goes; no pause. Cycles must be broken by hand with weak references.	Tracing garbage collector for managed code; COM reference counting for WinRT objects.
Path to native code and kernel	JNI and NDK into Bionic libc, then system calls.	Direct calls into libSystem, Mach traps and frameworks; no bridge.	P/Invoke and WinRT ABI (COM vtables) to Win32/NT.
Start-up time	Cold start pays for process fork, class loading and early interpretation; AOT/profile reduces it.	Lowest: native code, only dyld linking.	Low for AOT-compiled; higher for JIT.
Memory footprint	Highest: runtime, JIT code cache and GC headroom.	Lowest: no VM, no JIT cache, no GC headroom.	Medium.
Portability	Highest: the same DEX runs on any CPU architecture.	Lowest: ARM64 binary per architecture.	Medium: IL portable, AOT output not.

Conclusion for a low-memory device: ahead-of-time compilation with reference counting (the iOS design) is the most suitable. It has no JIT code cache, no tracing-GC headroom and no interpreter in memory, and it frees memory at once, so total footprint is smallest and nothing pauses the UI. The price is lower portability and the programmer's duty to avoid reference cycles. On Android the nearest equivalent is ART with full ahead-of-time compilation (speed filter) and a small heap.

3.4 Desk-check: cold start versus warm start

Step	Cold start (process absent)	Warm start (process alive, activity gone or stopped)	Reason
1 Request to AMS	Executed	Executed	The user action is the same.
2 Resolve and check	Executed	Executed	AMS always resolves the Intent.
3 Ask zygote	Executed	Skipped	AMS finds the process already running.
4 Fork and sandbox	Executed	Skipped	The process and its sandbox exist.
5 ActivityThread attach	Executed	Skipped	The process is already attached.
6 bindApplication	Executed	Skipped	Runtime, class loader and Application object are in memory.
7 Activity callbacks	onCreate, onStart, onResume	onRestart, onStart, onResume if only stopped; onCreate again if the activity was destroyed	Callback depends on how far the activity was torn down.
8 First frame	Executed	Executed, faster because views and caches may remain	Rendering always happens.

3.5 Dependency analysis

Each step needs the products of the earlier ones. Step 4 needs kernel process and memory services (fork, copy-on-write pages, UID switch); without them no isolated process exists. Step 6 needs the runtime initialised and the code files mapped, because the Application object is an object of managed code. Step 7's framework callbacks need the Application object and the Binder link from step 5, because AMS sends the callback commands through it. Application code in onCreate runs last because it assumes classes are loadable, memory is allocatable and the window system can be reached. Reordering any pair breaks that chain, which is why the sequence is strictly ordered.

Task 4 – Security Forensics

4.1 Locate the defects

SecureNotes decision	**OS mechanism bypassed or unused**	**Why the OS cannot protect the user**
1. Requests every permission "for future use"	Permission model (least privilege, runtime consent)	The OS enforces exactly what the user grants. A bulk grant gives the app, and any code running inside it, access to contacts, storage, location and more. The OS cannot tell which permissions are needed.
2. Token saved in a file readable by other apps	Sandboxing (per-app UID file isolation) and secure storage (Keystore/Keychain)	The kernel enforces the file mode the app chose; a world-readable mode lets any other UID read it. The hardware-backed Keystore is not used, so the token is plain bytes.
3. Updates accepted from any source	Code signing (an update must carry the same signing key as the	The signature rule is applied only when the update goes through the package installer. An app-side updater that accepts a package or code without checking the signer skips the OS check

	installed app)	entirely.
4. Plain unencrypted connections	Transport protection (TLS, network security policy)	The OS provides TLS but does not force its use. Cleartext traffic is readable and modifiable by anyone on the path, and the OS has no way to authenticate the server.

4.2 Attack scenario (network attacker plus update injection)

An attacker sits on the same public Wi-Fi as the victim and exploits defects 3 and 4, with defect 1 amplifying the damage.

1. The victim opens SecureNotes on café Wi-Fi; the app logs in over HTTP and sends the login token in a request header.
2. **Confidentiality is first lost here:** the attacker's access point captures the first cleartext request and reads the token (defect 4).
3. The app then checks for an update over HTTP; the attacker answers with a modified package or code file in place of the real update.
4. **Integrity is first lost here:** the app installs or loads the update without verifying who produced it (defect 3).
5. The attacker's code now runs inside the SecureNotes sandbox holding every permission the user granted (defect 1), so it can read contacts and notes and upload them.

The OS did nothing wrong at any step: it enforced the sandbox, the grant and the file modes that the app asked for.

4.3 Repair the design (only the faulty decisions are changed)

Defect	Repair	OS mechanism relied on	Where it sits in Android architecture
1	Request only the permissions the features use, and ask at run time	Runtime permission model	Checked by AppOps and PackageManagerService in system_server (framework, L5); the final

	when the feature is first used.		access limit is enforced through UID and group membership in the Linux kernel (L1).
2	Store the token in app-private internal storage, encrypted with a key held in the Android Keystore.	App sandbox (UID, SELinux) and hardware-backed Keystore	File mode and SELinux domain enforced in the Linux kernel (L1); Keystore keys are kept by the keystore daemon backed by the secure hardware through the KeyMint HAL (L2).
3	Remove the app's own updater; updates are installed only through the system package installer.	Code signing (APK signature scheme, same-signer rule)	PackageManagerService (L5) verifies the signature and compares it with the installed signer before installing.
4	Use HTTPS with TLS 1.2 or later; set the network security config to forbid cleartext; pin the server certificate.	Transport protection	Network security policy in the framework (L5) and the TLS library (Conscrypt, a native library, L3) inside the app process.

4.4 Regression check

Legitimate behaviour to keep: the user taps "Import contacts" and agrees, and the app reads the contacts.

Step	**Original app**	**Repaired app**

Permission request	READ_CONTACTS already granted at install	READ_CONTACTS requested at the moment the user taps "Import contacts"
User agrees	Not needed	The system shows the dialog; the user selects Allow
App reads contacts	A Binder call to the Contacts provider; permission checked; data returned	The same Binder call; permission checked and found granted; data returned
Login and sync	Works over HTTP	Works over HTTPS with the same server API

The repairs change when the permission is granted, where the token is kept and how data travels. They do not change the contacts feature or login, so the legitimate behaviour is preserved and the only difference is that the user must agree first.

Task 5 – Power Management Reasoning

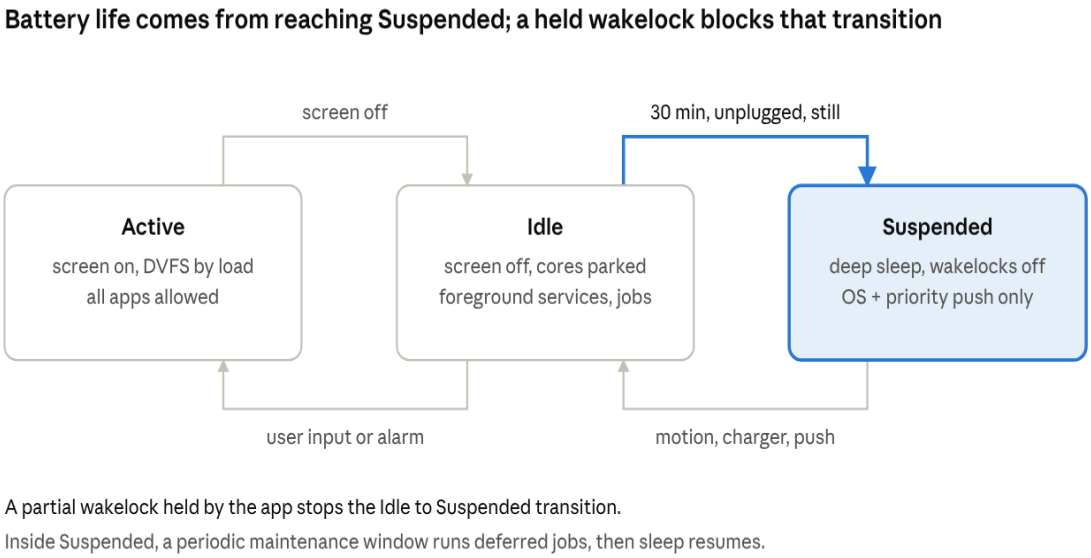
5.1 Power-state policy

The thresholds below are design values chosen for this policy.

State	**Hardware and kernel behaviour**	**Entry condition**	**Exit condition**	**Allowed to run**
Active	Screen on; CPU cores online; frequency chosen by the governor (DVFS) from load; radios on.	User input, incoming call, alarm, or charger connected.	Screen turns off (power button or screen timeout).	Foreground app, foreground services, all system services.
Idle	Screen off; display and GPU powered down; CPU in shallow idle	Screen off while the device is	User input returns to Active; screen	Foreground services with a notification, jobs

	states, cores mostly parked; radios on at a low duty cycle.	awake.	off for 30 minutes, unplugged and stationary moves to Suspended.	inside their constraints, held wakelocks, high-priority push handlers.
Suspended (deep sleep)	CPU in deepest idle or suspend-to-RAM; app wakelocks ignored; network access for apps blocked; only the modem and the sensor hub stay alive.	Idle for 30 minutes, battery power and no motion detected.	Motion detected, screen on, charger connected, high-priority push, or a periodic maintenance window.	Only OS services and a high-priority push handler; all other jobs and alarms are deferred to the maintenance window.

Transition	**Condition**
Active to Idle	Screen off
Idle to Active	Any user input or an alarm that turns the screen on
Idle to Suspended	30 minutes elapsed, no charger, no motion
Suspended to Idle	Motion, screen on, charger, or high-priority push
Suspended to Suspended (maintenance)	Periodic window opens, deferred jobs run, then sleep resumes



power-state model · 3 states, 4 transitions

Forward transitions run along the top and return transitions along the bottom; the shaded state is the one that saves battery.

5.2 Change the assumption – urgent push while suspended

What becomes unsafe or insufficient. The Suspended state blocks all app execution, so an urgent push cannot be handled. The Idle state allows held wakelocks, and the fitness app's permanent processor wakelock is exactly what stops the Idle to Suspended transition, which is why the battery drained within hours.

How a platform mechanism solves it. Push-based wake-up: the device keeps a single shared connection to the push service, owned by a system process, so the app needs no connection of its own. The modem wakes the processor only when a message arrives, and a high-priority message gives the app a short temporary allowance to run, then the device sleeps again. Sensor readings that would have needed the processor awake are collected by the sensor hub using hardware batching (a latency window is set when registering the listener) and are delivered to the app in a group when the processor wakes anyway.

What the application must stop doing. It must stop holding a partial wakelock permanently, stop polling with repeating timers or exact alarms, and stop keeping its own socket open in the

background. Deferrable work goes to scheduled background tasks (for example WorkManager), which the OS runs inside maintenance windows.

5.3 Change the requirement – battery saver at 15% charge

Level	What changes when battery saver activates
Kernel	The maximum CPU frequency is capped and high-performance cores are parked or kept for the foreground only; background tasks are confined to efficiency cores; the idle timer to deep sleep is shortened.
Runtime	Background compilation and optimisation work is postponed until the device charges; background threads of non-foreground apps get lower priority and fewer wake-ups.
Application	The app is notified of power-save mode; it lowers the sampling rate, batches readings and uploads, and defers non-essential syncing to the charging window.

Trade-off. Step and activity data on screen may be several minutes behind real time until the user opens the app. This is acceptable because, at 15% charge, keeping the device usable for calls and messages matters more than live statistics, and no data is lost: batched readings are shown as soon as the app is opened.

Task 6 – Design Challenge: Rural Care Field-Data Collection

Stage 1 – Interpret the problem

Element	Statement
Input	Patient records typed in by field workers on three device classes: A (modern Android), B (iPhone), C (J2ME/WML feature phones), often with no connectivity for hours.
Output	Every record stored once, unaltered, on the central server; unreadable to anyone who finds a lost device.

Objective	Collect patient data for a ten-hour shift on a single charge, and synchronise without loss or duplication when connectivity returns.
Constraints	Offline for hours; ten-hour battery; confidentiality if a device is lost; no loss and no duplicates after sync; Class C has only a J2ME runtime or a WML browser.
Decisions to make	Which application type per class; where and how records are stored offline; how data is encrypted at rest and in transit; when the radio is used; how sync resumes after an interruption.

Stage 2 – Strategy

Option	**Assessment against the constraints**
One cross-platform codebase for all classes	Cannot reach Class C (J2ME). For A and B it adds a bridge or a JavaScript engine, which costs memory and battery and gives indirect access to Keystore/Keychain.
WML or web fallback for Class C	A WML browser needs a live connection to load and submit a page; it has no local storage. It fails the "offline for hours" constraint for data capture.
Restrict to smartphones	Excludes Class C workers, who are the field workers with the least infrastructure.
Native per platform plus a shared sync protocol	Each class gets the runtime and security hardware it actually has, and one server protocol keeps data identical.

Chosen strategy: an offline-first native application on each class: a native Android app (A), a native iOS app (B) and a J2ME MIDlet (C), all talking to one server through the same

idempotent sync protocol. WML is not used for capture; it is kept only as an online read-only fallback.

Why it suits the problem. Offline capture needs local persistent storage on every class: SQLite on A and B and the MIDP RecordStore (RMS) on C. Lost-device protection needs hardware-backed key storage, which only native code reaches directly (Android Keystore, iOS Keychain/Secure Enclave). The ten-hour battery needs control over radio and wake-ups, which WorkManager (A) and BackgroundTasks (B) give; on C the radio is used only when the worker presses Send. Because the protocol is shared, the three apps differ in storage and power code but not in correctness logic.

Stage 3 – Critical information

Item	**Class A (Android)**	**Class B (iPhone)**	**Class C (J2ME)**
Runtime environment	ART, DEX with ahead-of-time and JIT	Native ARM64 binary, Objective-C/Swift runtime, ARC	CLDC/MIDP virtual machine (interpreter)
Security mechanisms	UID sandbox, SELinux, hardware-backed Keystore, screen lock, signed APK	App sandbox, Keychain and Secure Enclave, Data Protection file encryption, mandatory code signing	MIDP sandbox and signed MIDlet only; no hardware secure storage
Power-management controls	Doze, App Standby, WorkManager constraints, battery saver, sensor batching	Background task scheduler, Low Power Mode, background refresh limits	None programmable; only closing the app and controlling the backlight
Offline storage	SQLite (Room), encrypted	SQLite or Core Data with Data Protection	RMS RecordStore, small device-dependent size
Connectivity	Intermittent 4G or Wi-	Intermittent 4G or Wi-Fi	GPRS/2G,

	Fi		HTTP/HTTPS or SMS

Information the system must retain to prevent loss or duplication

Retained item	**Why it is required**
record_id (UUID created on the device)	Identifies a record forever, so the server can ignore a record it has already stored (no duplicates).
device_id and per-device sequence number	Orders records and shows gaps, so a missing record is detected.
Record state: NEW, SENT, ACKED	Shows what has been confirmed; a record is deleted only after ACKED, so nothing is lost on an interrupted sync.
Last server-acknowledged sequence number	The restart point after an interrupted sync.
Payload hash (SHA-256)	Detects corruption in transfer or storage.
Retry count and last attempt time	Controls retry back-off and so saves battery.
Worker identity and key identifier	Allows authenticated upload and the right key for decryption on the server.

Stage 4 – Complete design specification

4.1 Architecture per device class

Class	**Layers used**	**Runtime**	**Framework and components**
A Android	Linux kernel, Keystore HAL, ART, application framework, app	ART	Room database over encrypted storage; WorkManager sync worker; Keystore for keys; network security config (HTTPS only).

B iPhone	Darwin kernel, Core OS, Core Services, Cocoa Touch, app	Native binary with ARC	Core Data/SQLite with complete Data Protection; Keychain for keys; BGProcessingTask sync; App Transport Security.
C J2ME	Phone OS, CLDC/MIDP, MIDlet	CLDC virtual machine	RecordStore for records; MIDP HTTP connection for sync; lightweight crypto library inside the MIDlet.

4.2 Data capture and offline storage

Field	**Content**
record_id	UUID v4 generated on the device when the form is saved
device_id, seq_no	Registered device identifier; sequence number that increases by one per record and is never reused
created_at	Device time at capture, used for display only, never for ordering
payload	Patient data, encrypted before it is written
payload_hash	SHA-256 of the encrypted payload
state	NEW, SENT or ACKED

A record is written in a single transaction (Class A, B) or a single RecordStore write (Class C), and the form shows "Saved" only after that write is confirmed. Only a patient code is stored, not the full identity.

4.3 Security design

Area	**Class A**	**Class B**	**Class C**
Data at rest	AES-256-GCM; key in hardware-backed Keystore, usable only after user	AES under Data Protection (complete protection,	AES-128 applied inside the MIDlet; key derived from the worker PIN (PBKDF2), never stored

	authentication	locked when the device locks); key in Keychain, accessible only when unlocked	
Data in transit	HTTPS TLS 1.2+, certificate pinning, cleartext forbidden	HTTPS TLS 1.2+, certificate pinning	HTTPS where the handset supports it; otherwise the payload is already encrypted with AES and carries an HMAC, so cleartext transport still protects content
If the device is lost	App locks after 2 minutes idle; PIN or biometric needed; 10 wrong attempts delete the key; the server revokes the device token and the organisation sends a remote wipe	Same locking and key deletion; remote wipe and token revocation	MIDlet locks after 2 minutes; 5 wrong PINs delete the record store; server revokes the device token on report

4.4 Power policy for a ten-hour shift

Rule	Class A and B	Class C
Screen and display	Screen off after 30 seconds idle; low brightness default	Backlight off after 15 seconds idle
Location and sensors	One location fix per record, never continuous tracking	None

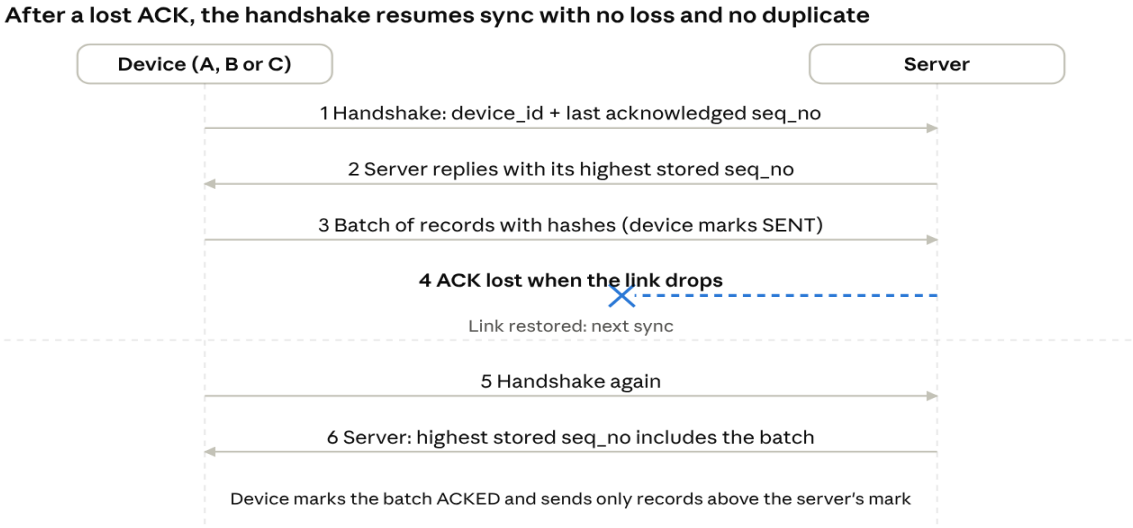
Radio while offline	No polling and no retries; sync tasks do not start until a network constraint is true	Radio not used until the worker presses Send
Sync trigger	Network connected, battery above 30% or charging; batches of up to 20 records; at most one sync every 30 minutes	Manual Send command; batches of up to 5 records
Battery 20% and below	Background sync stops; capture continues; sync only on charger or manual command	Same: manual only
Battery 10% and below	Capture-only mode: minimal UI, no animations, no sync except manual flush	Same

4.5 Synchronisation procedure

1. A trigger fires (constraint met on A and B, or the Send command on C).
2. The device authenticates to the server over TLS with its device token.
3. The device sends device_id and its last acknowledged seq_no; the server replies with the highest seq_no it has stored for that device.
4. The device selects records with a greater seq_no (states NEW or SENT), in seq_no order, up to the batch size.
5. The device sends the batch with a batch_id and every payload_hash, and marks the records SENT; nothing is deleted.
6. The server checks authentication and each hash, then stores each record by record_id (a record already stored is skipped), commits the batch in one transaction and replies ACK with the highest contiguous seq_no.
7. On ACK the device marks those records ACKED; ACKED payloads are removed only after the next successful sync cycle.
8. Steps 4 to 7 repeat until no NEW or SENT records remain; the device stores the sync time.

Interrupted sync. If the link drops before the ACK arrives, the records stay SENT. The next sync starts again at step 3, where the server states what it holds, so the device resends only the

missing records. The server skips any record_id it already stores, so nothing is duplicated. A batch with a failed hash check is rejected and resent.



sync with an interrupted ACK · device and server

The server committed the batch but the device never saw the ACK; the second handshake tells the device what the server holds, so nothing is resent twice and nothing is lost.

4.6 Failure handling

Failure	Class A	Class B	Class C
App or phone crashes mid-write	Database transaction rolls back; the record is not shown as saved	Same	The RecordStore write is atomic per record; the form is re-entered
Storage full	Stop capture with a warning; remove only ACKED payloads	Same	Warn at 90% of the store; require a sync; remove ACKED records only
Battery reaches the limit	Capture-only mode as in 4.4	Same	Warn the worker to send and power down

No secure hardware	Not applicable	Not applicable	Application-layer AES with a PIN-derived key, data minimisation and wipe after 5 wrong PINs
Server unreachable	Back-off by retry count; records stay queued	Same	Manual retry; records stay in the store

Stage 5 – Desk-check: Class A, offline six hours, battery 20%, phone lost

Step	**State and decision**	**Protective action**
1. Capture, hours 0 to 6	No network: the sync constraint is false, so no radio wake-ups. Records enter state NEW, encrypted with the Keystore key.	Each record is encrypted before storage.
2. Battery reaches 20%	Battery policy moves to capture-only; background sync stays off.	Energy is kept for data entry.
3. Phone is lost, screen locks	The app locks after 2 minutes idle; the Keystore key needs user authentication again.	Without the PIN or biometric, the key cannot be used.
4. Finder tries to open the app	Wrong PINs are counted; 10 failures delete the key.	The ciphertext becomes permanently unreadable.
5. Report and revocation	The organisation reports the loss; the server revokes the device token and sends a remote wipe.	Any later connection by the device is refused or wiped.

Exposed: no readable patient data, provided the phone was locked; only metadata (the app exists, the database size) is visible. **Lost:** the records of the six hours that were never synchronised, because the only copy was on that device. **Recoverable:** everything synchronised earlier in the shift is on the server; if the phone is found before the wipe, all six hours are intact. The design protects confidentiality but cannot recover unsynchronised data, which is why opportunistic sync at any connectivity is part of the policy.

Stage 6 – Attack your own design

Adverse condition	Why it is challenging	Prediction	Result
1. Stolen unlocked phone	The OS is already unlocked, so Keystore authentication is satisfied and the database can be read.	Data is exposed during the unlocked window.	The window is limited: the Keystore key requires re-authentication after 60 seconds and the app locks after 2 minutes, so exposure is bounded but not zero. **Critical decision:** key validity time plus app-level lock. The assumption "the phone is locked when lost" fails; refinement: remote wipe on report and storing only patient codes.
2. Corrupted or cut sync mid-transfer	The server may have committed a batch while the device never received the ACK, which can cause loss or duplication.	No loss and no duplicate.	Confirmed. At the next handshake the server's highest seq_no marks those records as stored; any resend of a stored record_id is skipped; a corrupted batch fails the hash and is resent. **Critical decision:** idempotent record_id and the handshake high-water mark. Refinement for app-data reset: the new install gets a new device_id, and record_id still prevents duplicates.
3. Class C device with no secure storage	A PIN-derived key can be attacked offline if the record store is copied off the handset; a four-digit PIN has only 10,000 possibilities.	The design is weaker on Class C.	Not fully defeated, but not safe against a determined attacker. **Failed assumption:** that the PIN protects the store. Refinements: require a longer passphrase (at least 8 characters), keep only patient codes and the minimum fields on Class C, and shorten retention by syncing and deleting ACKED records at every opportunity.

Conclusion. The design meets the requirements for Classes A and B and for sync integrity on all classes. Conditions 1 and 3 expose real limits, so the critical decisions are key-use time limits, data minimisation and the idempotent record_id sync protocol; the remaining Class C risk is accepted and reduced by the refinements above.

ASSIGNMENT 4B

How to read this part. Assignment 4B asks for measured evidence from an emulator or device, which cannot be produced by writing alone. For each task this doc gives the experiment design, the exact commands or code, the prediction (made before running, as the task requires) and the interpretation. Lines marked **[capture]** are the outputs you must paste from your own run; no measured value is invented here. The security audit (Task 4) and the tracking prototype (Task 6) are Python programs that were actually run, and their outputs below are real.

4B Task 1 – Analyse and Validate a Mobile OS Model

1.1 Layer, process and IPC mapping

Item	**Android (evidence command)**	**iOS (documented source)**
Layers	Linux kernel, HAL, native libraries, ART, framework, apps. Evidence: adb shell getprop ro.build.version.release, adb shell uname -a, adb shell ls /system/lib64 /vendor/lib64, adb shell service list	Core OS, Core Services, Media, Cocoa Touch (Apple platform architecture documentation)
Process creation	adb shell ps -A -o PID,PPID,NAME shows zygote64 as parent of system_server and of every app process	launchd (PID 1) starts system daemons and launches apps through the app launch services (Apple documentation)
IPC	Binder: adb shell dumpsys activity services, adb shell service call for a live transaction, adb shell dumpsys binder_calls_stats	Mach ports and XPC services between apps, daemons and extensions (Apple documentation)
Lifecycle	Activity: onCreate, onStart, onResume, onPause, onStop, onDestroy; Service;	UIApplication states: not running, inactive, active,

	BroadcastReceiver. Evidence: adb logcat -s LifecycleTest from a test app	background, suspended; UIViewController: viewDidLoad, viewWillAppear, viewDidAppear, viewDidDisappear

[capture] the ps output with the zygote parent highlighted and the dumpsys activity processes header for your test app. iOS rows rely on documented sources, which the task allows when a device is not available.

1.2 Expose the OS behaviour

Commands: adb shell dumpsys activity processes | grep -E "ProcessRecord|oom|state", adb shell cat /proc/<pid>/oom_score_adj, adb shell dumpsys meminfo <package>.

Value shown	Meaning	Effect on keep or kill
oom_score_adj (0 foreground, about 100 visible, about 200 perceptible, 900 and above cached)	Importance of the process to the user	When memory is short, lmkd kills the highest score first, so cached background apps die before the foreground one.
Process state (top, service, cached)	What the framework believes the process is doing	Decides the adj value assigned.
PSS from dumpsys meminfo	Proportional memory the process actually uses	Among equal scores, large processes free the most memory when killed.

1.3 Requirement change – three conditions, predictions first

Test app: one Activity with a text field, a counter held in a ViewModel and a value saved in onSaveInstanceState, logging every callback with tag LifecycleTest.

Condition (how to create it)	Predicted survival	Predicted last callback	Predicted state on return
A. Foreground, screen on	Alive	onResume	Nothing to restore
B. Background 5 minutes (press Home)	Alive but cached, high oom_score_adj	onStop	Fully kept in memory, ViewModel and fields intact
C. Low memory: adb shell am send-trim-memory <pkg> RUNNING_CRITICAL, then process death with adb shell am kill <pkg>	Trimmed first, then killed	onStop (onDestroy is not called when a process is killed)	Only onSaveInstanceState data restored; the ViewModel counter and unsaved fields are lost

[capture] the logcat lines for each case and compare with this table. Interpret every mismatch, for example a process that survives the trim because the device had enough free memory.

1.4 Boundary test

Case: process death while a dialog is open over a form with unsaved input (open the dialog, press Home, run adb shell am kill <pkg>, return from Recents). It was chosen because state that lives only in memory (ViewModel, open dialog, background thread result) is lost, while the framework recreates the Activity as if nothing happened, so the user sees an empty form. It exposes whether the app stores state in SavedStateHandle or persistent storage.

4B Task 2 – Architecture Assumptions versus Measured Reality (9 marks)

Implementations

```
// Solution A: main thread
fun filterOnMain(pixels: IntArray) = blur(pixels, 0, pixels.size)


// Solution B: background executor, work split into chunks
suspend fun filterParallel(pixels: IntArray, parts: Int) = coroutineScope {
```

```
    val size = pixels.size / parts
    (0 until parts).map { i ->
        async(Dispatchers.Default) { blur(pixels, i * size, (i + 1) * size) }
    }.awaitAll()
}
```

Architecture for the responsive UI: MVVM. The Activity observes a StateFlow from the ViewModel; the ViewModel launches the work in viewModelScope on Dispatchers.Default; the repository holds the data; the main thread only draws.

2.1 Normal case

Workload: blur of a 4000 by 3000 image. Prediction: Solution A freezes the UI and drops frames for the full task time; Solution B finishes sooner and keeps frame deadlines. Evidence to capture: adb shell dumpsys gfxinfo <pkg> reset, run, then adb shell dumpsys gfxinfo <pkg> framestats (janky frames); a Perfetto trace with adb shell perfetto -o /data/misc/perfetto-traces/t.pftrace -t 10s sched freq gfx view and adb shell top -H -p <pid> showing worker threads spread over several cores.

2.2 Break the assumption

Workload: summing an array of 2,000 numbers, once on the main thread and once split over 8 threads with a single synchronized counter. Prediction: Solution B is slower, because the task takes microseconds while creating threads and acquiring the lock cost more than the work. **First design decision that causes the degradation:** splitting work finer than the cost of starting a thread, and funnelling every thread through one lock. **[capture]** the two durations, thread counts and context switches.

2.3 Engineering comparison

Metric	**Tool**	**Solution A**	**Solution B**
CPU time, thread count	Debug.threadCpuTimeNanos(), top -H	[capture]	[capture]
Context switches	/proc/<pid>/status (voluntary and	[capture]	[capture]

	nonvoluntary)		
Memory	dumpsys meminfo PSS, Debug.getNativeHeapAllocatedSize()	[capture]	[capture]
User-perceived quality	gfxinfo janky frames, input latency	[capture]	[capture]
Predictability	worst case and spread over 20 runs	[capture]	[capture]

2.4 Changed requirement – one charge for a full day

The design change to test: fewer threads (two), background thread priority (Process.THREAD_PRIORITY_BACKGROUND, which places the work in the background cpuset, normally the efficiency cores), and deferral through WorkManager with a charging or idle constraint. Expected result: a longer task time but fewer wake-ups and less energy. Your conclusion must quote your own run, for example the per-core time in the Perfetto trace and the energy estimate from dumpsys batterystats.

4B Task 3 – Runtime Behaviour under Changing Conditions

Benchmark workloads: (i) CPU loop (nested arithmetic over a large array), (ii) allocation-heavy routine (many short-lived objects), (iii) RecyclerView list of 10,000 items.

Stage A – Normal execution on ART

Item	**Command**
Cold start	adb shell am start -W -S <pkg>/.MainActivity (force-stop first, read TotalTime)
Warm start	Press Home, then adb shell am start -W <pkg>/.MainActivity
Compilation state	adb shell dumpsys package <pkg> (Dexopt state, status)
GC events and pauses	adb logcat -s art (concurrent copying GC lines with pause times)

Heap growth	repeated adb shell dumpsys meminfo <pkg> during the run

Stage B – Change the compilation mode

Commands: adb shell cmd package compile -m verify -f <pkg>, then -m speed-profile, then -m speed. Predictions:

Filter	**Most affected workload**	**Start-up**	**Steady-state**	**Storage**
verify (no AOT, interpreter then JIT)	CPU loop, which starts interpreted until JIT compiles it	Slower	Converges to similar speed once the JIT has compiled hot methods	Smallest
speed-profile	UI list and start-up path, compiled from the recorded profile	Faster than verify	Near full speed	Medium
speed (full AOT)	CPU loop from the first call	Fastest	Highest from the start	Largest compiled code

Roles: the interpreter runs everything first and records a profile; the JIT compiles hot methods while the app runs; profile-guided AOT compiles only the profiled methods when idle and charging; a Baseline Profile ships that profile with the app so first launch is already compiled. **[capture]** start-up times and the CPU-loop time for each filter.

Stage C – Production failure

Failure 1: a static field holding an Activity, plus an unbounded cache. Failure 2: a 10-second sleep on the main thread. Detection: LeakCanary reports the retained Activity; the GC log shows heap growth and rising pause counts until an OutOfMemoryError is thrown by the ART heap manager when the heap limit is reached; StrictMode.setThreadPolicy(...detectAll().penaltyLog()) logs the blocking call from the framework; the ActivityManager watchdog raises an ANR after

an input event is not handled within about 5 seconds and writes a trace to /data/anr. **[capture]** the leak report, the GC lines and the ANR trace.

Stage D – One code base for Android and iOS

Aspect	**Native ART (Kotlin)**	**Swift with ARC**	**Flutter (Dart AOT)**	**React Native or WebView hybrid**
Start-up	Depends on compile filter	Native binary, low	AOT, no JIT warm-up, plus engine start	JavaScript engine start and bundle load
Memory model	Tracing GC with pauses	Reference counting, no pauses	Generational GC tuned for frame budgets	JS engine GC plus native objects
UI thread	Main thread, Choreographer	Main thread	Own rendering engine, separate raster thread	JS thread, bridge, main UI thread
Native API access	Direct (JNI, framework)	Direct	Platform channels	Bridge or native modules

Recommendation: Flutter, if the product must have one code base. It is compiled ahead of time, so it avoids the JIT warm-up measured in Stage B; it renders itself on its own thread, so a blocked callback does not freeze frame production as seen in Stage C; and it has no JavaScript bridge. The costs are a larger app size and platform channels for native APIs. If your Stage A to C numbers show start-up as the dominant problem, native per platform remains the lowest-risk option.